TXELECTRUM

מגישים: מאיר קלמפנר ושחר סילון

מנחה: ברק עינב

הפקולטה להנדסת תכנה, מכון לב

התשפ"ב סמסטר ב

תוכן עניינים

תיאור הבעיה ופתרונות נוכחיים 3
מבנה המערכת 4
מבנה כללי 4
אלקטרום 5
Rest API 6
Host 7
פרוטוקול Host-Applet 8
Applet 9
תרשים רצף 10
היבטים אבטחתיים 11
עבודה עתידית 12
חלוקת עבודה 13
נספחים 14

תיאור הבעיה ופתרונות נוכחיים

הבעיה הגדולה ביותר של משתמשי ביטקוין, או משתמשי מטבעות קריפטוגרפיים באופן כללי, היא שמירת המפתח הפרטי שלהם. המפתח הפרטי הוא המהווה הוכחה לשייכות המטבעות, וכל מי שמחזיק בו יכול להשתמש במטבעות המשוייכים אליו.

נסקור להלן את הפתרונות הקיימים ואת היתרונות והחסרונות שלהם. כיום, ישנם 2 סוגים עיקריים של ארנקים, חמים וקרים:

ארנקים חמים:

- ארנק שולחני
  המפתח נשמר מוצפן בזיכרון הקשיח של המחשב.
  הבעיה בפתרון זה, היא שבזמן הרצת התכנה המפתח מפוענח, ובמידה שיש גישה לזיכרון ניתן לקרוא אותו.
- ארנק אינטרנטי
  המפתח נשמר באינטרנט. פתרון זה כמובן הוא הבעייתי ביותר. המשתמש סומך על צד שלישי שידאג לשמור את המפתח בצורה מאובטחת.

ארנקים קרים:

- ארנק תכנה / חומרה:
  בצורה זאת משתמשים ב2 ארנקים: אחד מחובר לרשת (מקוון) ליצירת טרנזקציות (להלן TX) ושליחתם לרשת, והשני מנותק מהרשת (לא מקוון) לחתימת TX. סדר הפעולות הוא כדלהלן:
  1. יצירת TX בארנק המקוון
  2. העברה לארנק הלא מקוון
  3. חתימה
  4. העברה לארנק הלא מקוון
  5. שליחה

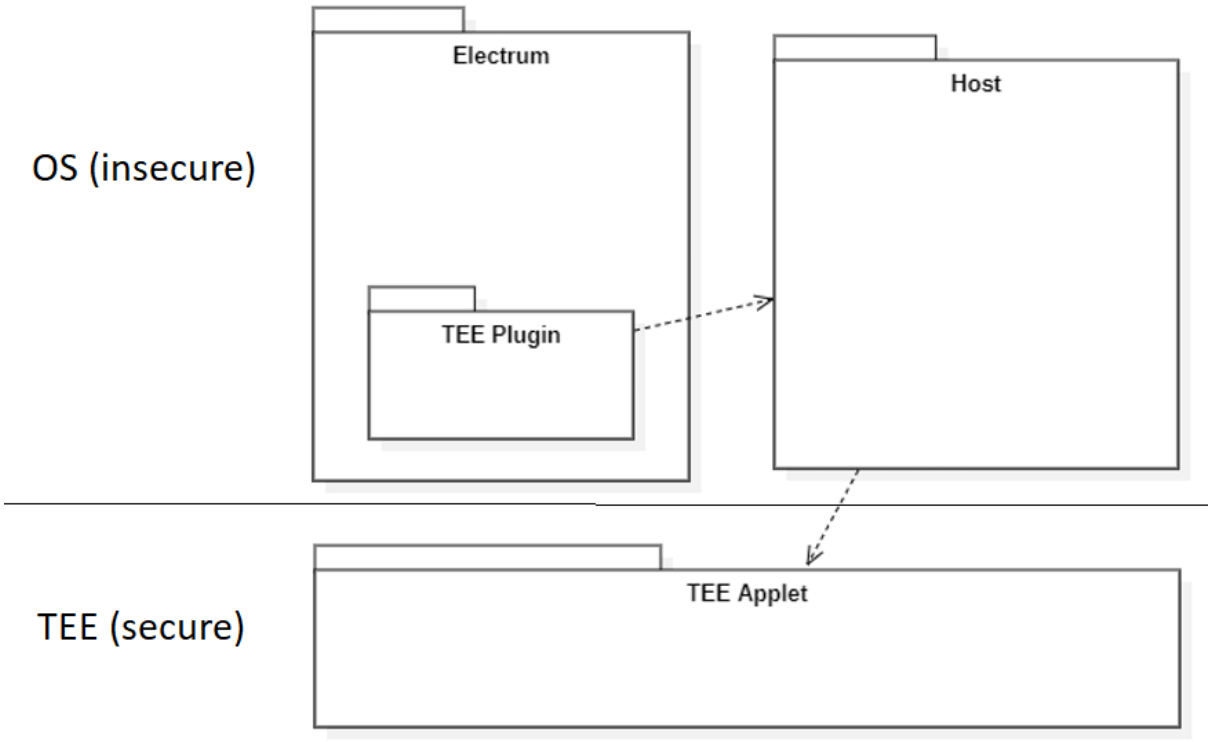
  בארנק תכנה הארנק הלא מקוון יכול להיות כל ארנק שהוא כל עוד המכשיר עליו רצה התכנה לא מחובר לרשת. בארנק חומרה הארנק הלא מקוון הוא חומרה ייחודית. פתרון זה הוא המאובטח ביותר, אולם השימוש מסורבל מאוד, בגלל הדרישה להחליף בין ארנקים.

הפתרון שאנו מציעים, הוא מימוש הארכיטקטורה של ארנק קר בעזרת סביבה בטוחה המובנית במחשב. בכך נשמר יתרון הארנק הקר, שהוא שמירה מאובטחת של המפתח, וגם נמנעים מהבעיה שלו, מכיוון שאין צורך לעבור בין ארנקים.

מבנה המערכת

המימוש שלנו הוא תוסף לארנק של אלקטרום, המשתמש בסביבה של אינטל, הנמצאת ברוב המחשבים.

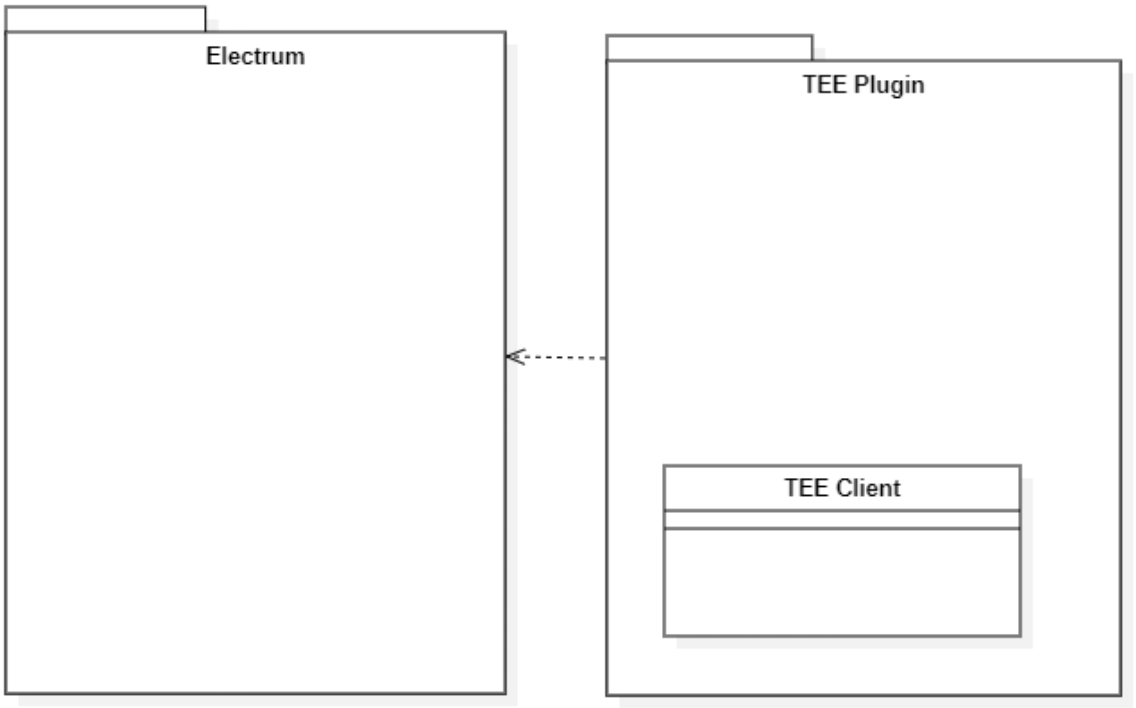
מבנה כללי



למערכת 3 חלקים עיקריים:

- הארנק של אלקטרום, בו נמצא גם התוסף לארנק
- Host, אחרי לקשורת עם הTEE מהצד של מערכת ההפעלה
- Applet, התכנה הרצה בTEE

אלקטרום



הארנק בו משתמשת המערכת הוא אלקטרום. בחרנו בו מכיוון שניתן לכתוב לו תוספים בצורה נוחה ופשוטה. הארנק והתוסף כתובים בפייתון.

התוסף מבצע שינויים ביצירת הארנק, כדי להתאימו למבנה הנדרש. כמו כן הוא בונה את הTX בעזרת הTEE ושולח אותם לרשת. הארנק בו משתמשים הוא מסוג imported (address).

המודול TEE client מתקשר עם השרת בRest API.

Rest API

הAPI כולל 2 בקשות: חילול זוג מפתחות, וחתימה. תיעוד לAPI נמצא כאן.

לשם חילול המפתחות שולח הלקוח סיסמה, איתה יאומת לאחר מכן כשיבקש לחתום על מידע.

בתשובה הוא מקבל את המפתח הפומבי, ממנו נוצרת לאחר מכן כתובת הארנק.

Create key pair

POST http://localhost:51841/create_keypair

Request

Body

hashed_password string<binary>
SHA-1 hash binary
>= 40 characters <= 40 characters
Example: 9f4470548199 75f30a80e593961d850788edf9b4
Match pattern: ^[A-Fa-f0-9]{40}$

Responses

200 400 500

OK

Body

public_key string<binary>
Representation of secp256k1 public key in SEC's compressed format.
>= 66 characters <= 66 characters
Example: 02f2dd109034c408433042ab0d237436ec62b5475ad87121ab607503c7d67d0895
Match pattern: ^[A-Fa-f0-9]{66}$

בשביל לחתום על TX, שולח הלקוח בקשה עם 3 פרמטרים: המידע עליו חותמים, המפתח הפומבי של המפתח הפרטי איתו חותמים, וגיבוב של הסיסמה להזדהות.

Sign a buffer

POST http://localhost:51841/sign_buffer

Request

Body

public_key string<binary>
Representation of secp256k1 public key in SEC's compressed format.
>= 66 characters <= 66 characters
Example: 02f2dd109034c408433042ab0d237436ec62b5475ad87121ab607503c7d67d0895
Match pattern: ^[A-Fa-f0-9]{66}$

hashed_password string<binary>
SHA-1 hash binary
>= 40 characters <= 40 characters
Example: 9f4470548199 75f30a80e593961d850788edf9b4
Match pattern: ^[A-Fa-f0-9]{40}$

hashed_buffer string<binary>
SHA-256 hash binary
>= 64 characters <= 64 characters
Example: 315f5bdb76d078c43b8ac0064e4a0164612b1fce77c869345bfc94c75894edd3
Match pattern: ^[A-Fa-f0-9]{64}$

Responses

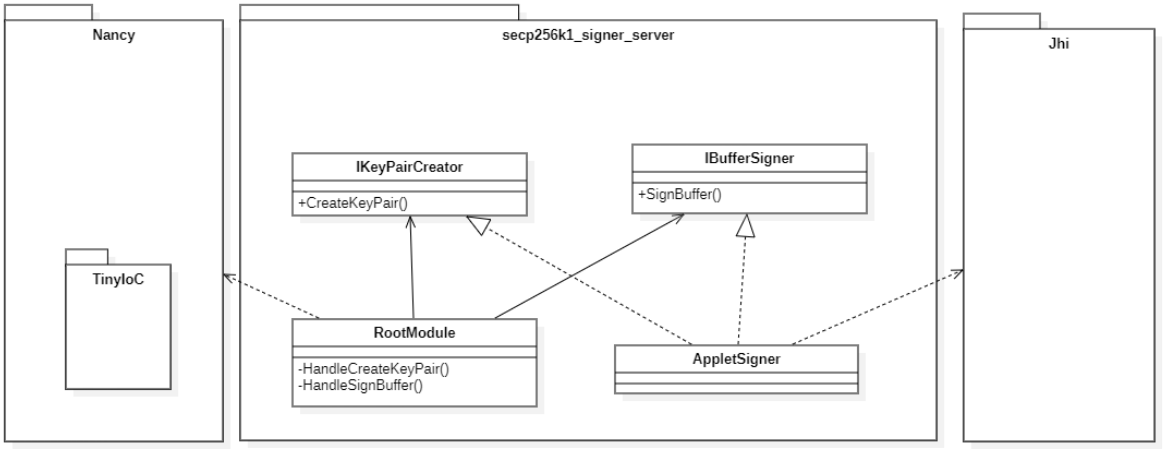
200 400 401 404 500

OK

Body

signature string<binary>
Representation of secp256k1 digital signature in DER format.
Example: 3045022100f8a4dbab9c9fed17dfe75db206c2c070f86ae8537b87a221906bf6c981dd1259 6aab19c0efda4c0a965b1a6ceb5
Match pattern: ^[A-Fa-f0-9]*$

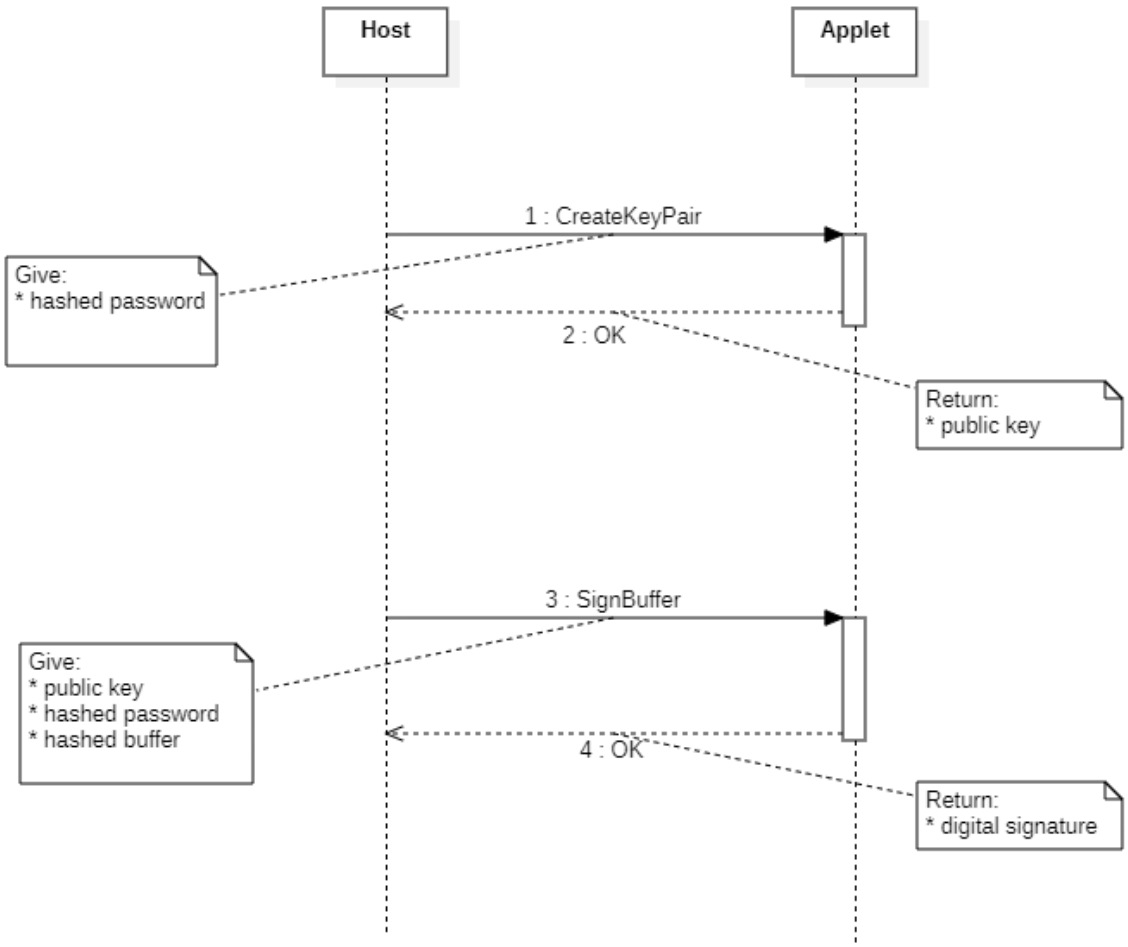
Host



בHost יש שרת HTTP המקבל את בקשות הלקוח, מעביר אותן לApplet ומחזיר לו את התשובות מהApplet.

- השתמשנו בספריה Nancy ליצירת השרת.
- השתמשנו בספריה TinyIoC, שהיא ספריה מובנית בNancy, למימוש Dependency Injection.
- המחלקה RootModule אחראית על מימוש הREST API באמצעות הספריה Nancy.
- המחלקה AppletSigner אחראית על מימוש הפרוטוקול עם הApplet (פירוט בעמוד הבא) באמצעות הספרייה Jhi, והיא מוזרקת לתוך הRootModule באמצעות TinyIoC.

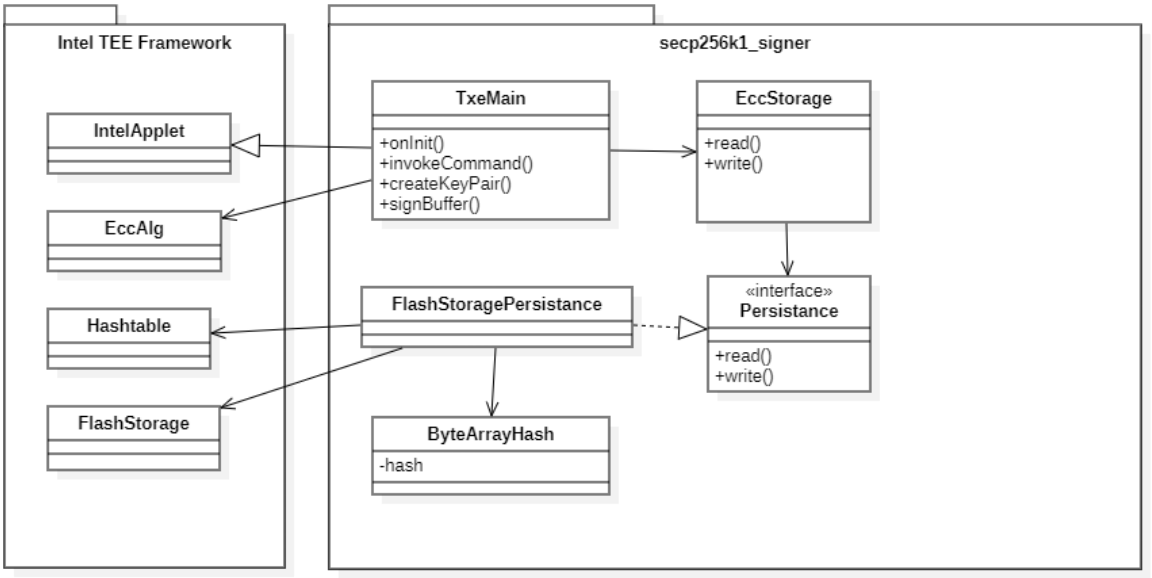
פרוטוקול Host-Applet



לאחר שהHost מקבל בקשה מהלקוח, הוא ממיר אותה לפורמט הקריא על ידי הApplet, לפי הפרוטוקול המוגדר בניהם.

הסבר מפורט בGithub.

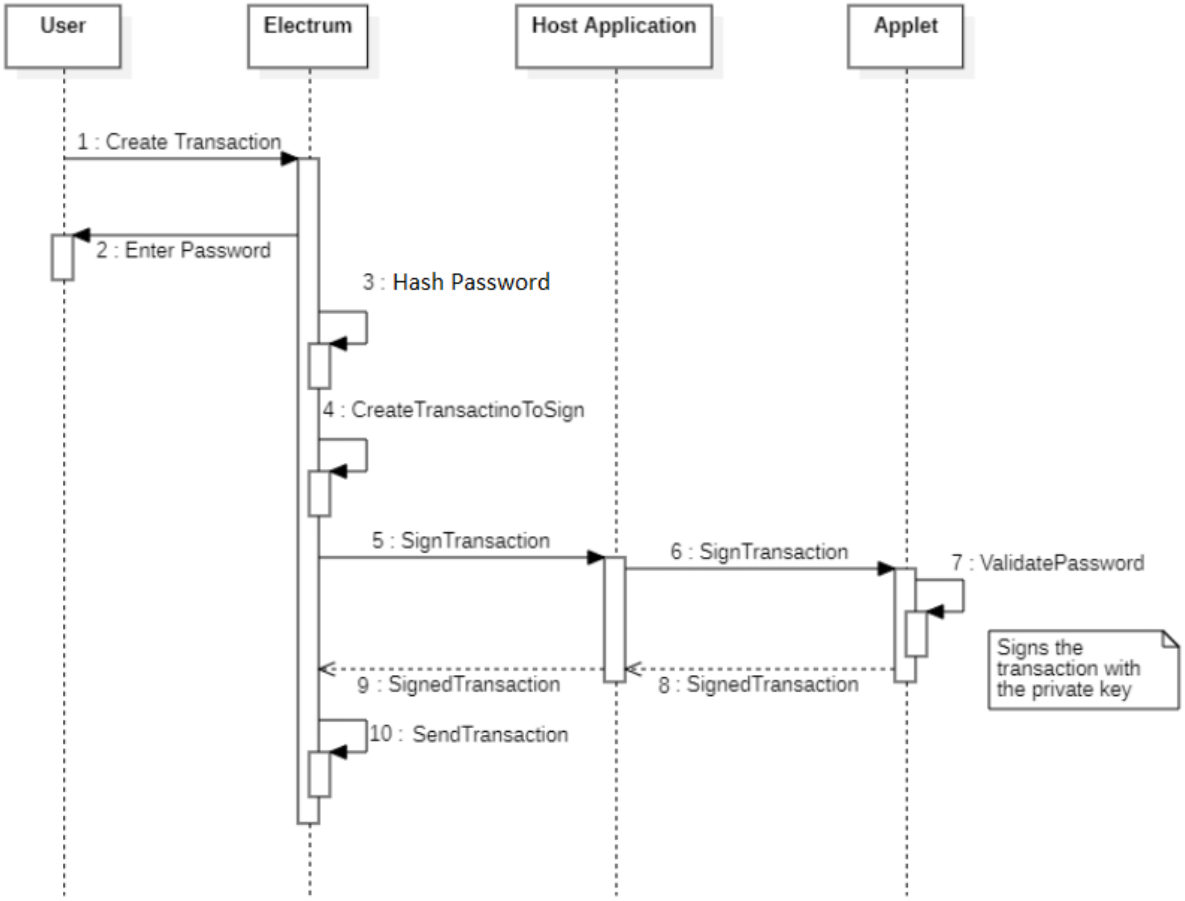
Applet



הApplet רץ בסביבה הבטוחה של אינטל. בפרויקט השתמשנו באימולטור של הסביבה.

- TxeMain: מממש את הפרוטוקול בין הHost והApplet.
- EccStorage: אחראי על שמירת ואחזור המפתח הציבורי והמידע. המפתח הציבורי מאוחזר לפי המפתח הפרטי.
- FlashStoragePersistance: שומר את המידע (כמערך של בתים) על הFlashStorage, ומאחזר אותו לפי מפתח (גם מערך של בתים).
- ByteArrayHash: מגבב מערך של בתים לפי התוכן, ליצירת מפתחות לטבלת גיבוב.

תרשים רצף



1. המשתמש יוצר TX באלקטרום
2. המשתמש מכניס סיסמה
3-4. יוצרים את הTX ומגבבים את הסיסמה
5-6. ההודעה נשלחת לHost והוא מעביר אותה לApplet
7. במידה והסיסמה נכונה, Applet חותם על הTX
8-9. הTX החתומה נשלחת לאלקטרום
10. הTX נשלחת לרשת

היבטים אבטחתיים

ההישג העיקרי של הפתרון שלעיל מבחינת אבטחה, הוא שמירה על המפתח הפרטי בסביבה הבטוחה, כאשר באף שלב הוא לא יוצא ממנה, מוצפן או גלוי. הדבר מונע קריאה שלו בכל דרך שהיא, מן הזכרון הקשיח או בזמן ריצה.

בנוסף, מכיוון שבמקרה ה(כמעט*) הכי גרוע, רק הסיסמה תיגנב, יהיה אפשר לגנוב מטבעות רק על ידי חתימת TX בתוך המחשב עצמו. הדבר מוריד מעט מחומרת המצב, מכיוון שכל עוד המחשב לא מחובר לרשת המשתמש יוכל להשתמש בארנק כארנק קר רגיל.

אולם, הפתרון שלעיל אינו מושלם, בעיקר מכיוון שפרוטוקול ההזדהות אינו מאובטח דיו.

- התקשורת בין הלקוח לTEE לא מאובטחת. תוכניות נוספות יכולות בתנאים מסויימים להאזין לתקשורת.

- אין אימות של הTEE מבחינת הלקוח. כלומר, הלקוח לא יכול לדעת אם הוא אכן מדבר עם הTEE או עם מתחזה. הדבר יכול להוביל לגניבה של הסיסמה, *או במקרה יותר חמור, השרת שמלכתחילה יצר את המפתחות הוא מתחזה.

- ניתן לגנוב את הסיסמה בעזרת שימוש ברישום הקשות, או מקריאת זיכרון הריצה של תכנת הארנק.

עבודה עתידית

אבטחה

השיפורים הדרושים מבחינת אבטחה:

- הוספת secure display להכנסת הסיסמה
- אבטחת התקשורת בין הלקוח לTEE עם SIGMA וEPID

ביטקוין

יש לעקוב אחר השינויים בפרוטוקול הביטקוין ולעדכן לפי הצורך. העדכונים הדחופים הם:

- BIP32: ארנקים היררכיים, המאפשרים שימוש בכמה כתובות תחת מפתח-על אחד
- BIP39: אחזור המפתח הפרטי בעזרת seed

UX

תהליך יצירת הארנק מסורבל במידה מסוימת, מכיוון שהפרויקט נכתב כתוסף ולא כחלק אינטגרלי מהמערכת. יש להפוך את תהליך יצירת הארנק לדומה ככל הניתן לתהליך ליצירת ארנקים מסוגים אחרים.

באופן כללי, כדאי לשנות את המערכת כך שלא תהווה תוסף, אלא להוסיף סוג חדש של ארנק לאלקטרום. הדבר אמנם ידרוש יותר עבודה, אך ישפר באופן ניכר את חוויית המשתמש ויפשט את מימוש המערכת.

חלוקת עבודה

מאיר כתב את השרת והTEE client (API) והApplet (יצירת מפתח וחתימה)

שחר כתב את התוסף לאלקטרום (יצירת ארנק חדש, יצירת TX ושליחה שלה)

נספחים

הסבר מפורט כיצד להפעיל את המערכת

https://github.com/silon1/txelectrum#readme

קוד המקור של הפרויקט

https://github.com/silon1/txelectrum

קוד המקור של אלקטרום

https://github.com/spesmilo/electrum